High Performance Programming

Lecture 8

"Distributed memory programming with MPI"

Lecturer: Ramoni Adeogun

AALBORG
UNIVERSITET

Teaching plan

Parallel concepts, models and algorithms

- ~~Introduction to parallel processing (RA) - 13/02/2025~~
- ~~Parallel processing: Basic data structures and communication (RA) – 20/02/2025~~
- ~~Paradigms for parallel programs (RA) – 27/02/2025~~
- ~~Analytical modelling of parallel systems and applications (RA) – 06/03/2025~~

High performance programming, hardware architecture verification

- ~~Practical aspects of parallel programming (SNE) – 13/03/2025~~
- Distributed memory parallel programming (RA) – 20/03/2025
- Shared memory parallel programming (SNE) – 27/03/2025
- GPU programming & Vectorization (SNE) – 03/04/2025
- Testing and verification (RA) – 24/04/2025

Workshop: Design and implementation of high-performance programs – 01/05/2025

- Problems released – 27/03/2025
- Report submission – 08/05/2025

Contents

1. Basics of distributed memory programming

Distributed memory basics

Advantages vs. shared memory programming

- **Programs can run** in shared memory systems
- **Distributed memory programming is the only option** for distributed systems
- **Good** for multiple similar tasks that take a long time
- **Scalability: unlimited power**

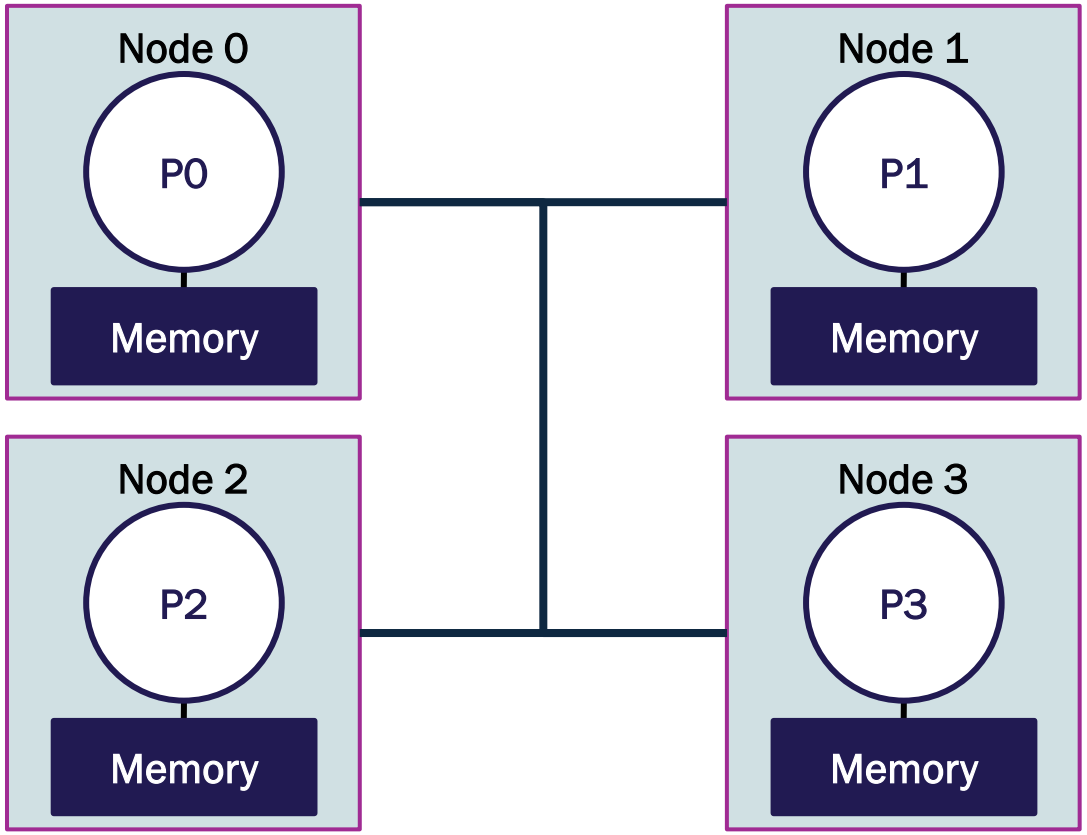
Disadvantages

- Data must be transmitted explicitly
- Communication overhead is a major concern
- Processes occur asynchronously but there are **some instructions** to synchronize
- Data must be protected

The paradigm of distributed memory systems

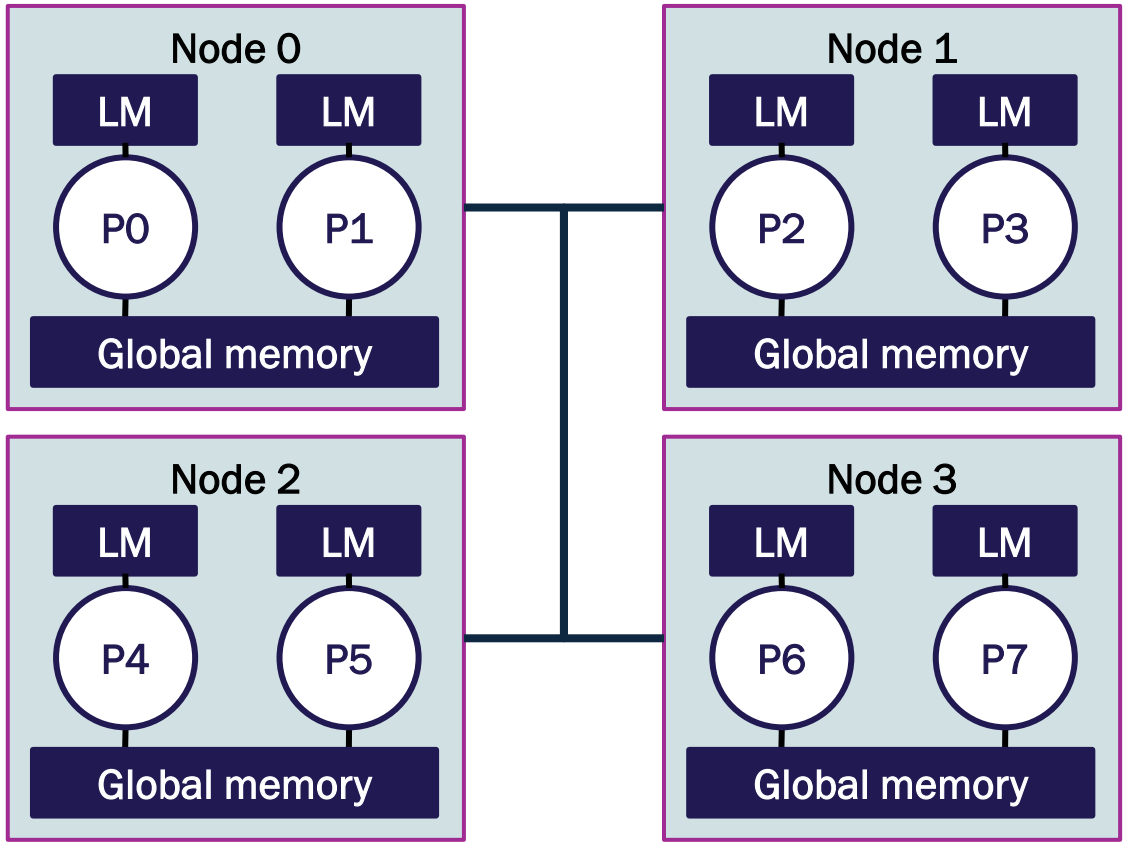
Assumption: Fully distributed memory

Each node has a single processor



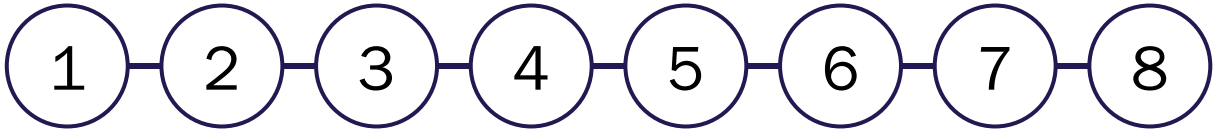
Reality: Hybrid architecture

Each node contains multiple CPUs (link)



Revisiting the network model

Graph $G = (V, E)$



Each node $u \in V$ is a processor

Each edge $\{u, v\} = \{v, u\} = uv \in E$ is a communication link

We need additional constructs to **send**(m, u) and **receive**(m, v)

Processors must pause the execution to send or receive data

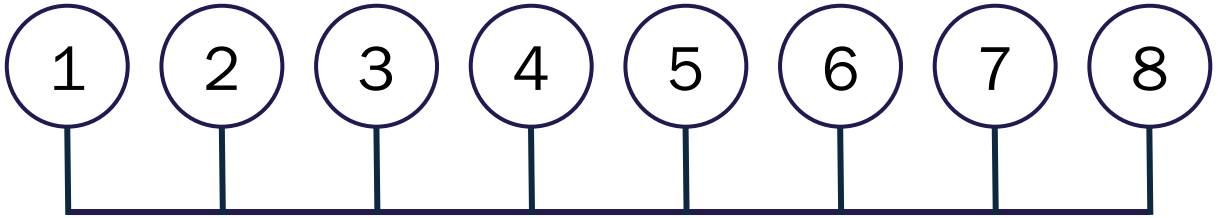
The greatest challenge is to find out how to distribute the data

What is the real communication model?

Revisiting the network model

Graph $G = (V, E)$

Each node $u \in V$ is a processor



Cut-through routing

Each edge $\{u, v\} = \{v, u\} = uv \in E$ is a communication link

We need additional constructs to **send**(m, u) and **receive**(m, v)

Processors must pause the execution to send or receive data

The greatest challenge is to find out how to distribute the data

What is the real communication model?

Communication costs

Interactions between processes on different nodes require **message passing**

How do we move the data efficiently?

Cut-through routing, single-port communication, bidirectional and equal links

Communication cost $t_{\text{comm}} = t_s + mt_w$ between any pair of processors

Startup time t_s

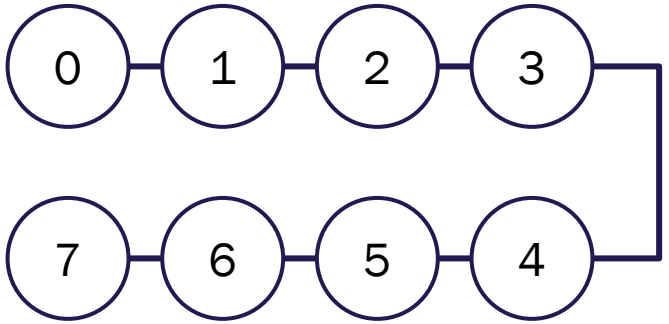
Message length in words m

Per-word transfer time t_w

Where is the time to traverse the edges? Propagation time

Basic network topologies

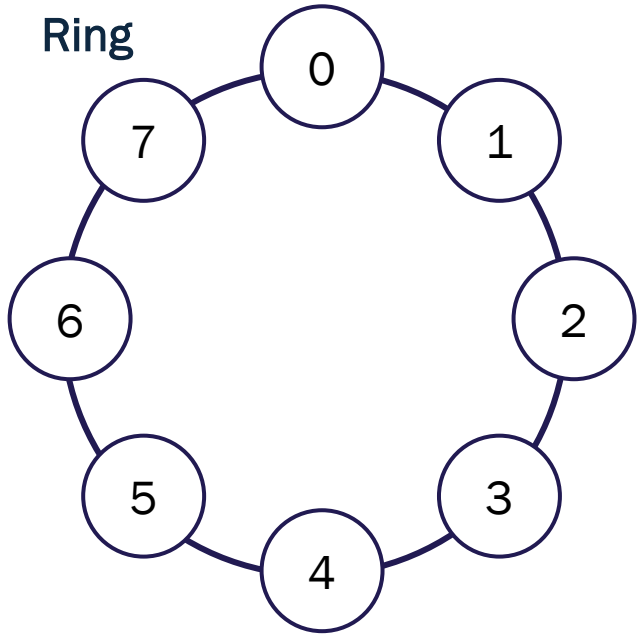
Linear



Diameter: $p - 1$

Degree: 2

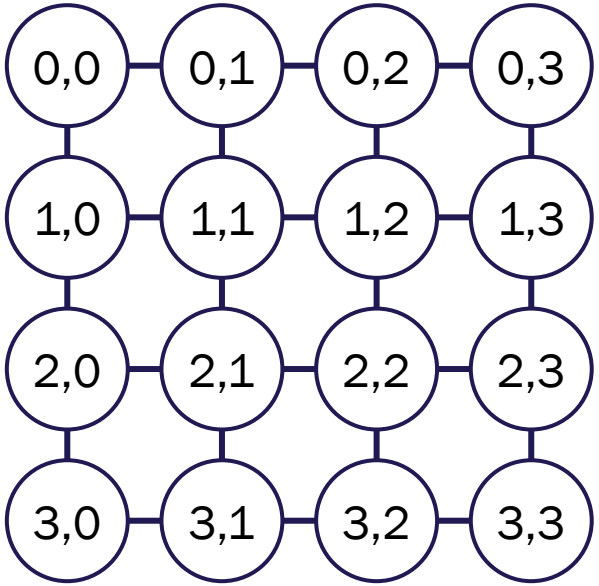
Ring



Diameter: $\lfloor p/2 \rfloor$

Degree: 2

Mesh

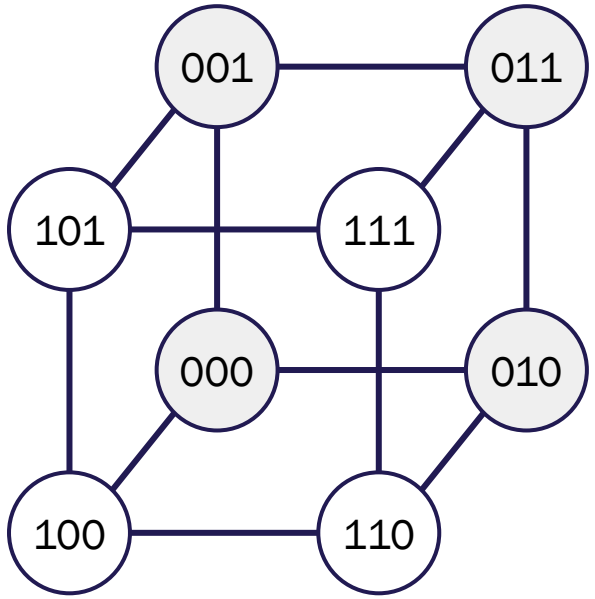


Diameter: $2\sqrt{p} - 2$

Degree: 4

Basic network topologies

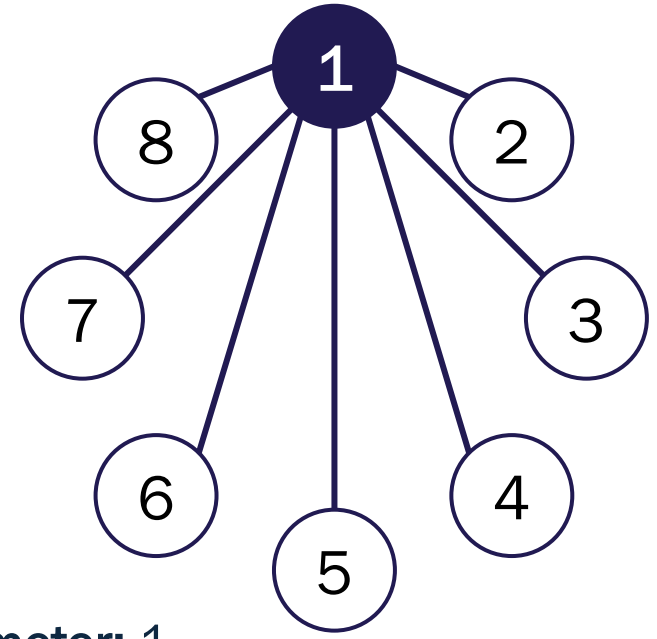
Hypercube



Diameter: $\log_2 p$

Degree: 6

Fully connected (not scalable)



Diameter: 1

Degree: $p - 1$

Common topologies in supercomputers

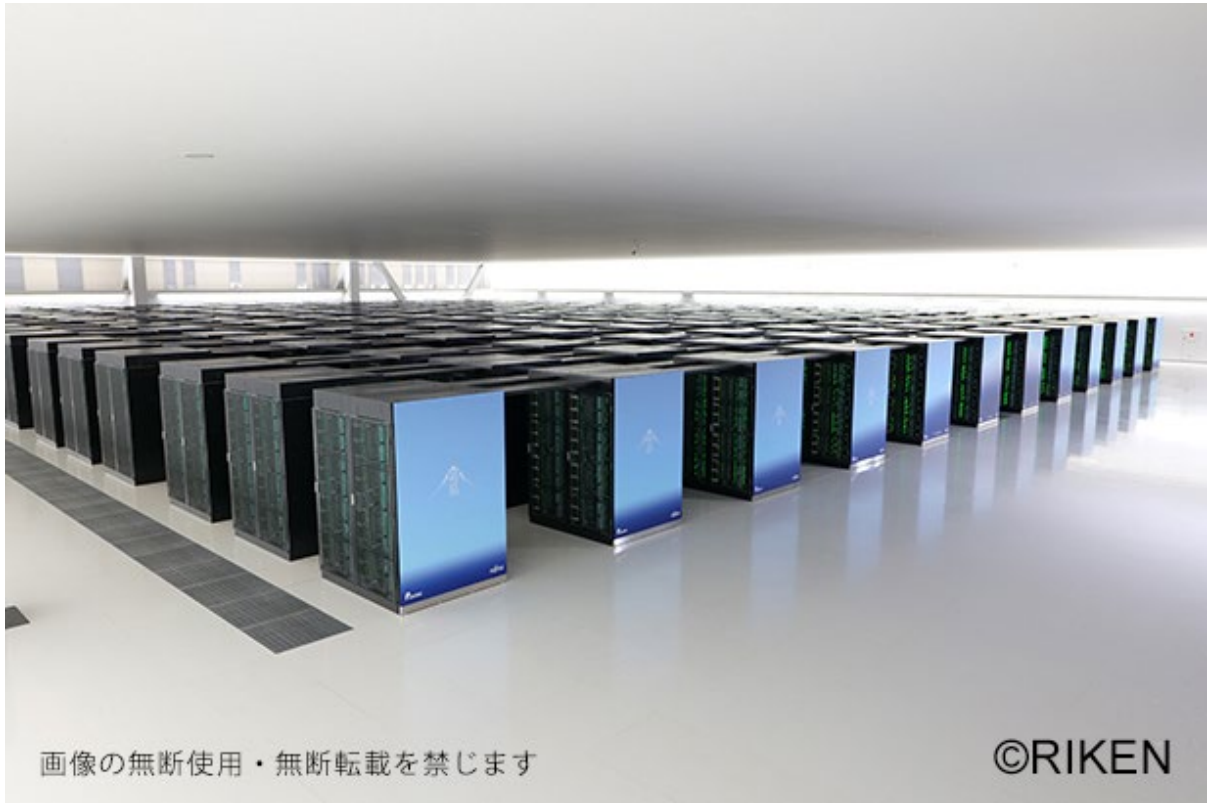
El Captain

Frontier

Aurora

Eagle

Fukagu



Credit: Photo gallery RIKEN center for computational science

Common topologies in supercomputers

El Captain, Eagle, Frontier: Dragonfly

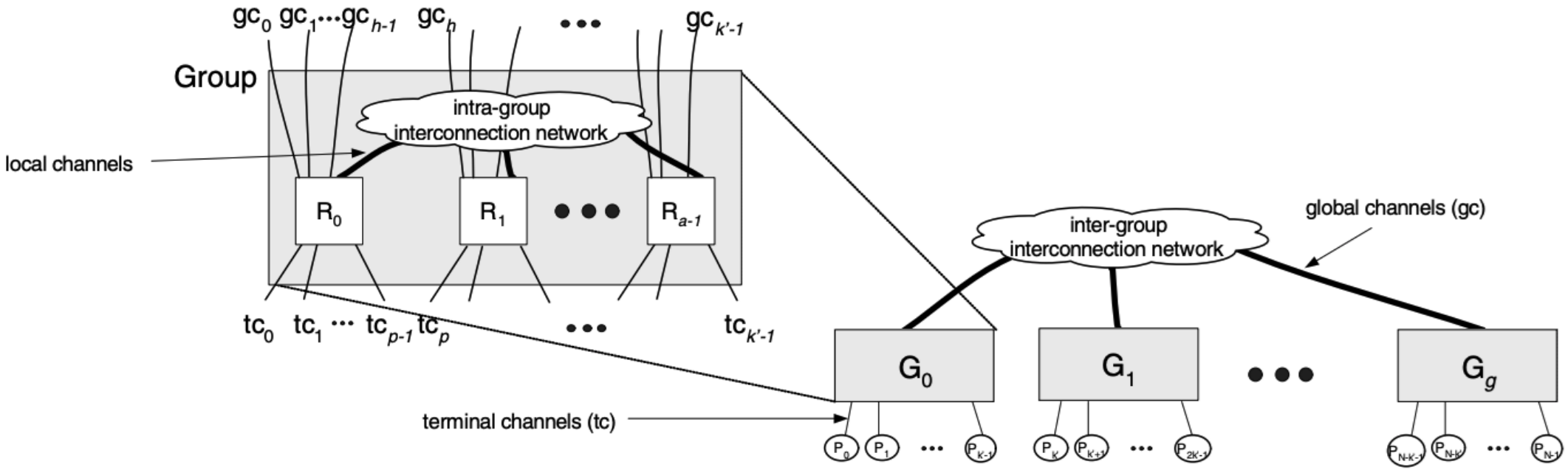


Image credit: J. Kim, W.J. Dally, S. Scott, and D. Abts,"Technology-Driven, Highly-Scalable Dragonfly Topology," International Symposium on Computer Architecture, pp. 77-88, 2008.

Common topologies in supercomputers

Aurora: Fat-tree topology

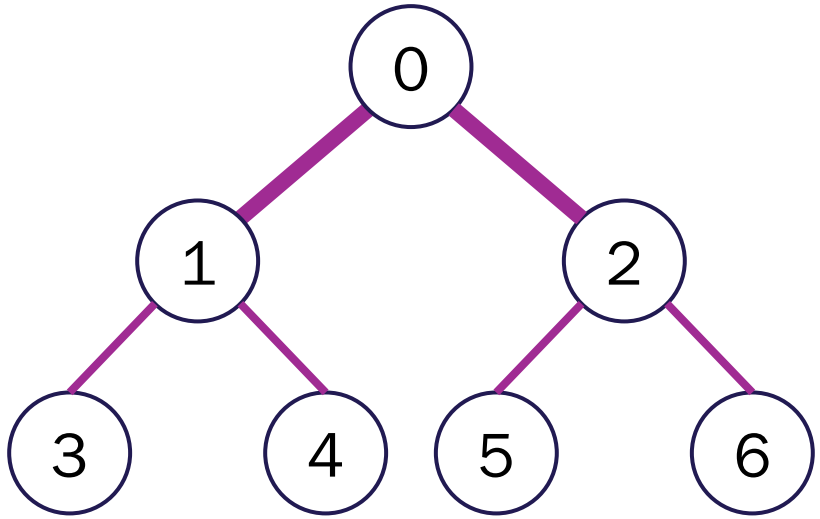


Image credit: Flickr

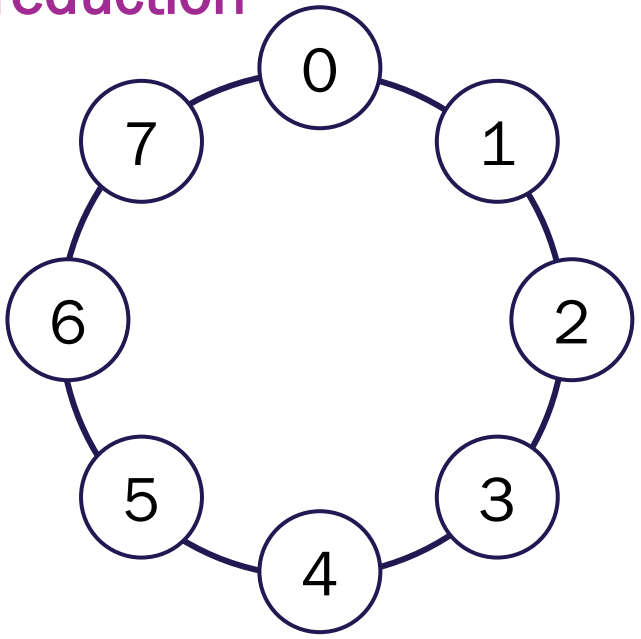
Communication operations

Operation

- Send
- One-to-all broadcast
- Scatter
- All-to-all broadcast

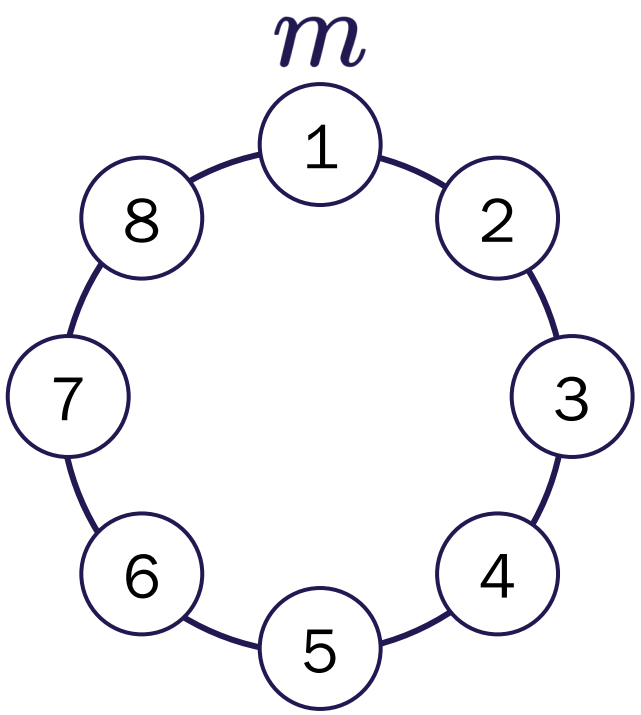
Dual

- Receive
- All-to-one reduction
- Gather
- All-to-all reduction

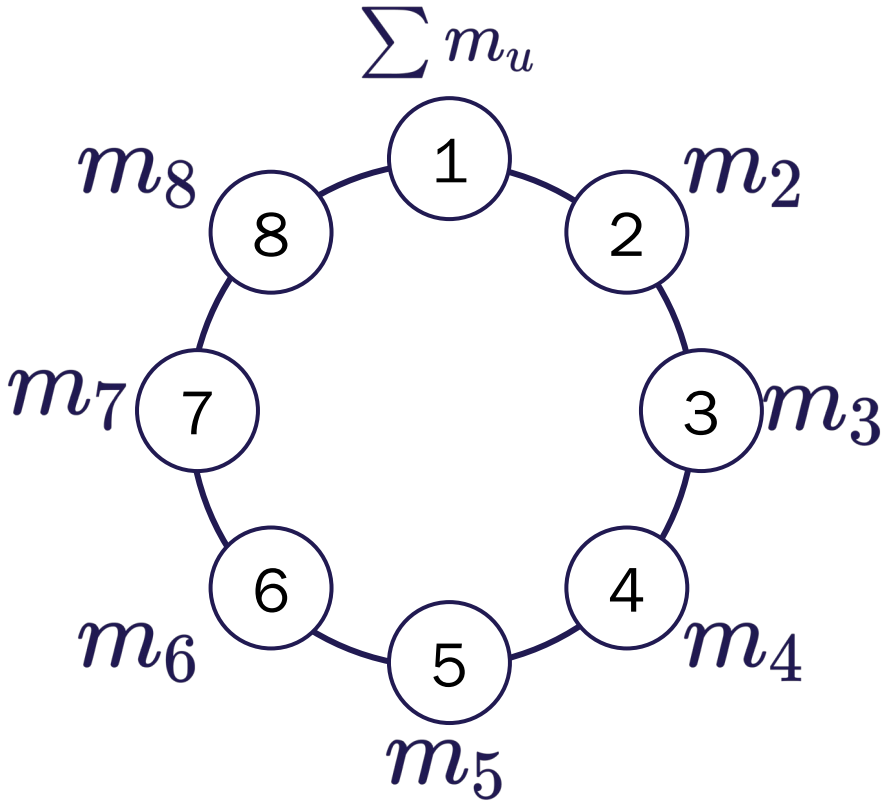


Dual communication operations

One-to-all broadcast

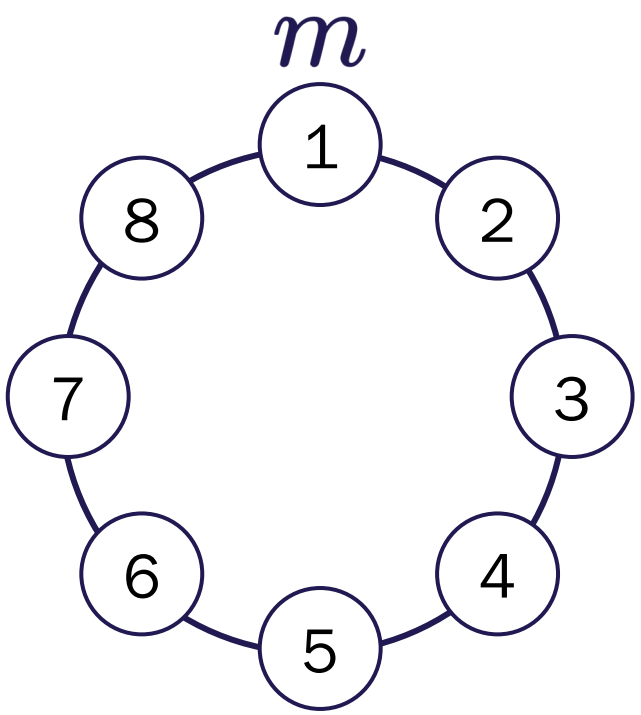


All-to-one reduction

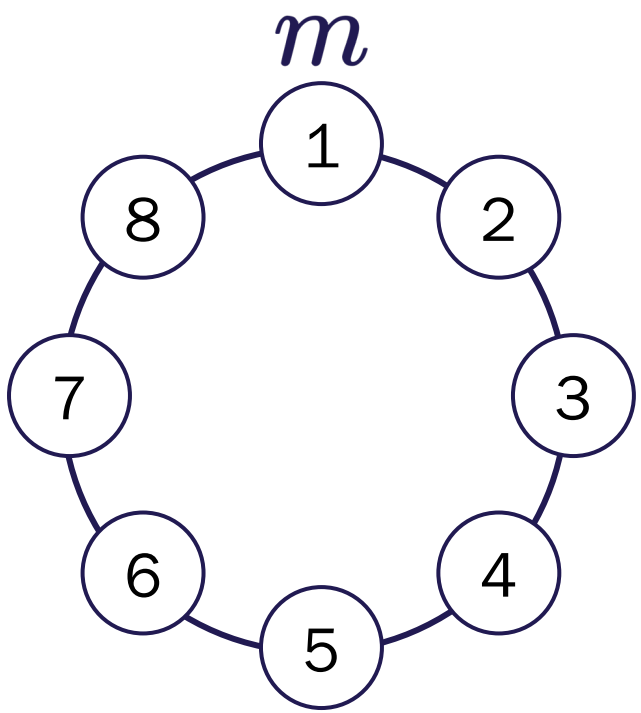


One-to-all broadcast in ring and linear topologies

Naïve algorithm: $p - 1$ steps

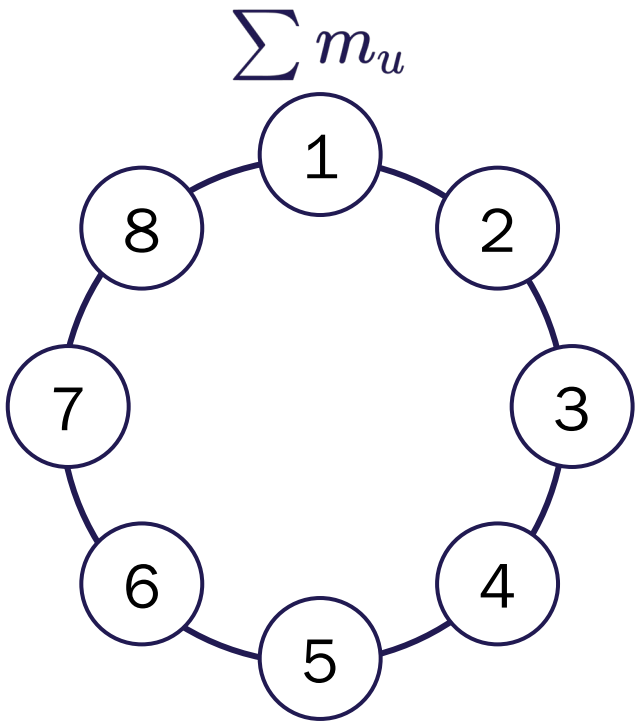


Recursive doubling: $\log_2 p$ steps

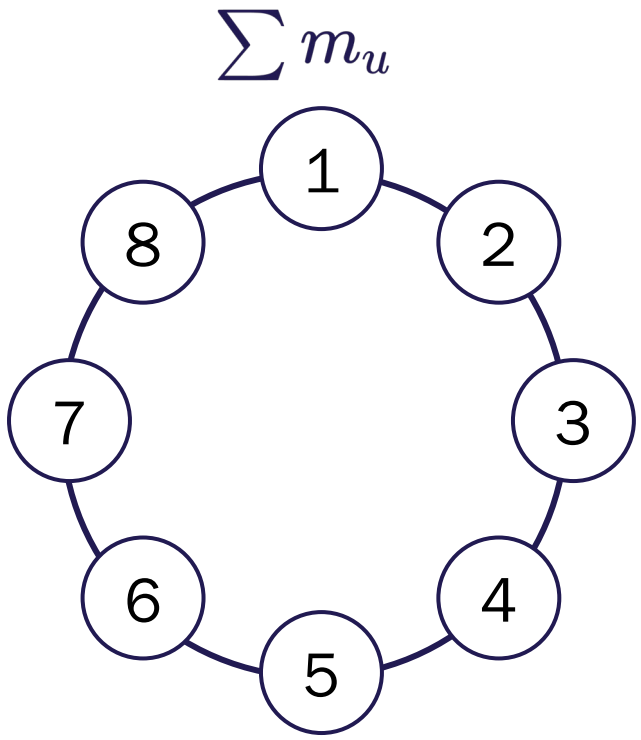


All-to-one reduction in ring and linear topologies

Naïve algorithm: $p - 1$ steps

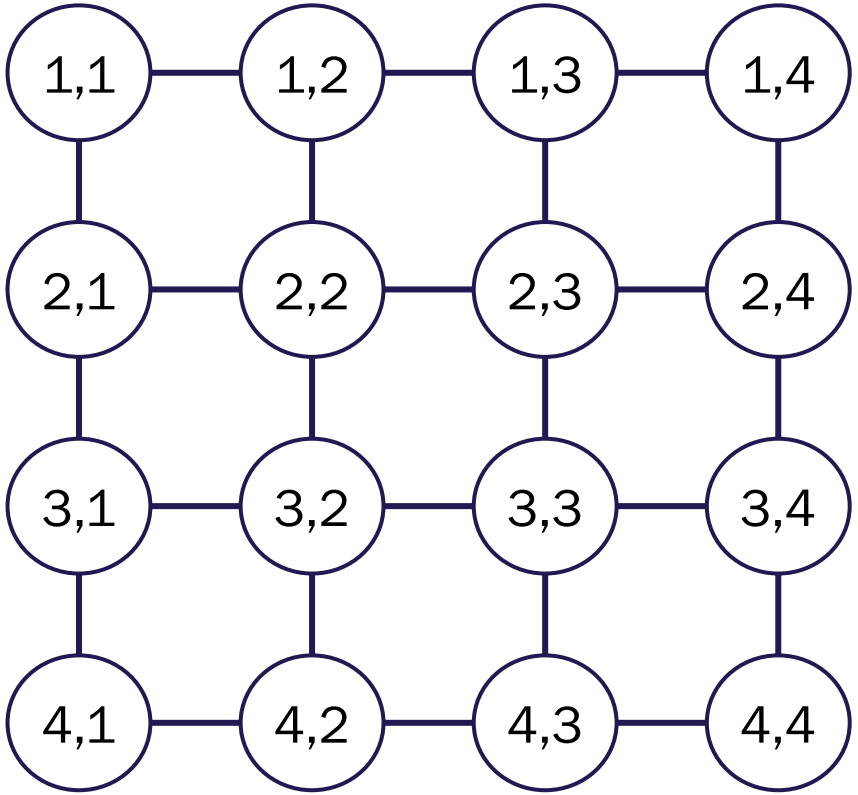


Recursive doubling: $\log_2 p$ steps



One-to-all broadcast in mesh topologies

Recursive doubling in two phases of $\log_2 \sqrt{p}$ steps



Hence, this requires a total of $2 \log_2 \sqrt{p} = \log_2 p$ steps

2. The Message Passing Interface (MPI)

Who is Jack Dongarra?

Recipient of the 2021 ACM Turing Award

Developed LINPACK and MPI

"These libraries have now been written for single processors, parallel computers, multicore nodes, and multiple GPUs.

His software libraries are used, practically universally, for high performance scientific and engineering computation on machines ranging from laptops to the world's fastest supercomputers."

–Forbes

Dongarra and Summit, Credit: ACM

Message Passing Interface (MPI)

Standardized and portable message-passing system

Designed to function on a wide variety of parallel computers.

Link to Windows download and setup

Provides essential virtual topology, synchronization, and communication functionalities.

Communicator

- Point-to-point and collective communications
- Blocking and non-blocking communication

Run MPI with:

```
mpiexec -n p python3 script.py
```

The mpi4py module

Instead of creating processes inside the code, we distribute the work by using conditionals

We'll use the communicator instance COMM_WORLD

In the communicator, each process has a rank used to define its behavior

```
from mpi4py import MPI

comm = MPI.COMM_WORLD

rank = comm.Get_rank()

if rank == 0:                                   # This is the main process

        Do something

else:                                           # The other processes

        Do something else
```

Communications

Point-to-point

Have an associated tag to identify the messages at the receiver end

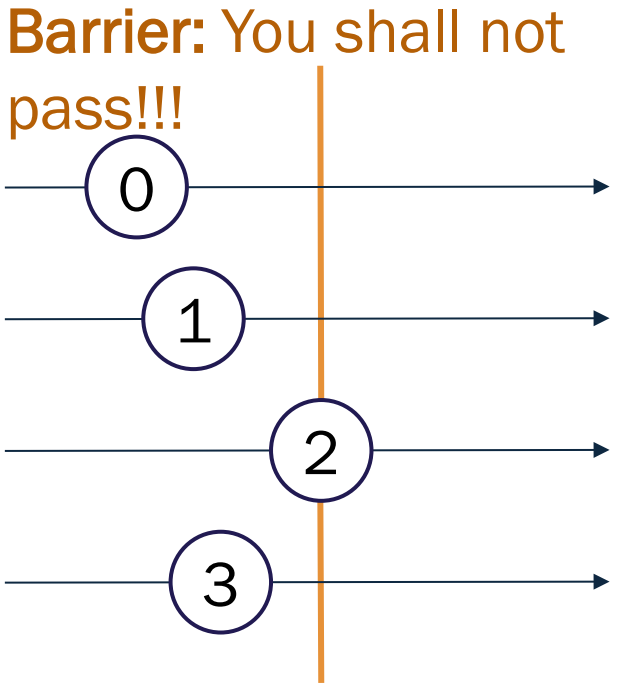
- Send
- Receive

Collective communications

Transmit data between multiple processes of a group

Messages have no tag associated

- Barrier, to synchronize the processes
- Broadcast
- Gather
- Scatter
- Global reduction operations, such as sum (we saw this in ufuncs)



Simple example from MPI4PY tutorial

```
from mpi4py import MPI

comm = MPI.COMM_WORLD                          # Create the communicator

rank = comm.Get_rank()

if rank == 0:

    data = {'a': 7, 'b': 3.14}

    comm.send(data, dest=1, tag=11)   # Send data specifying a destination and tag

elif rank == 1:

     data = comm.recv(source=0, tag=11)
```

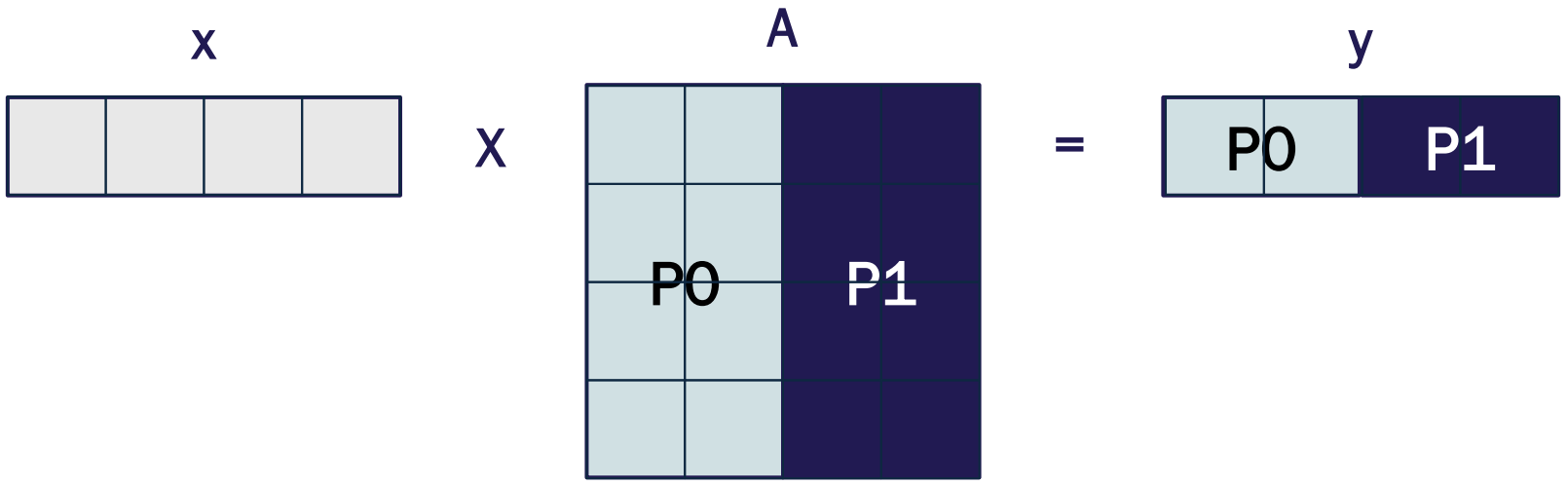
In recv: source identifies the transmitter and tag identifies the message. Can be left in blank

3. Applications and examples

Parallel vector-matrix multiplication

The operation can be easily parallelized

Which communication operations do we need?



Hyperparameter tuning in Machine Learning

"One of the most important breakthroughs in reinforcement learning was the development of an off-policy [temporal-difference] TD control algorithm known as **Q-learning"**

– Sutton and Barto, "Reinforcement Learning: An Introduction," 2015

An agent learns from experience, without a model of the environment

We need:

- **A set of states:** Where is the agent in the system?
- **A set of actions:** What can the agent do at a given state?
- **A reward:** How good is an action at a given state? Not known until the action is chosen
- **A Q-table:** Which action is the best at each state?
- **Hyper-parameters:**
  - How fast the agent learns α,
  - How much it takes into account past experiences γ
  - How frequently it explores other options ε

The Taxi environment from OpenAI GYM

Four designated locations in the grid world: R(ed), G(reen), Y(ellow), and B(lue).

The taxi starts off at a random square and the passenger is at a random location.

The taxi drives to the passenger's location, picks up the passenger, drives to the passenger's destination, and then drops off the passenger.

Once the passenger is dropped off, the episode ends.

There are 6 discrete deterministic actions:

- Move: north, south, east, west
- Drop-off or pick up

Rewards: -1 for movement, -10 for failed pickup/dropoff, and +20 for successful trip

Even “bigger” applications

Cloud computing

Speed of light is 300000 km/s

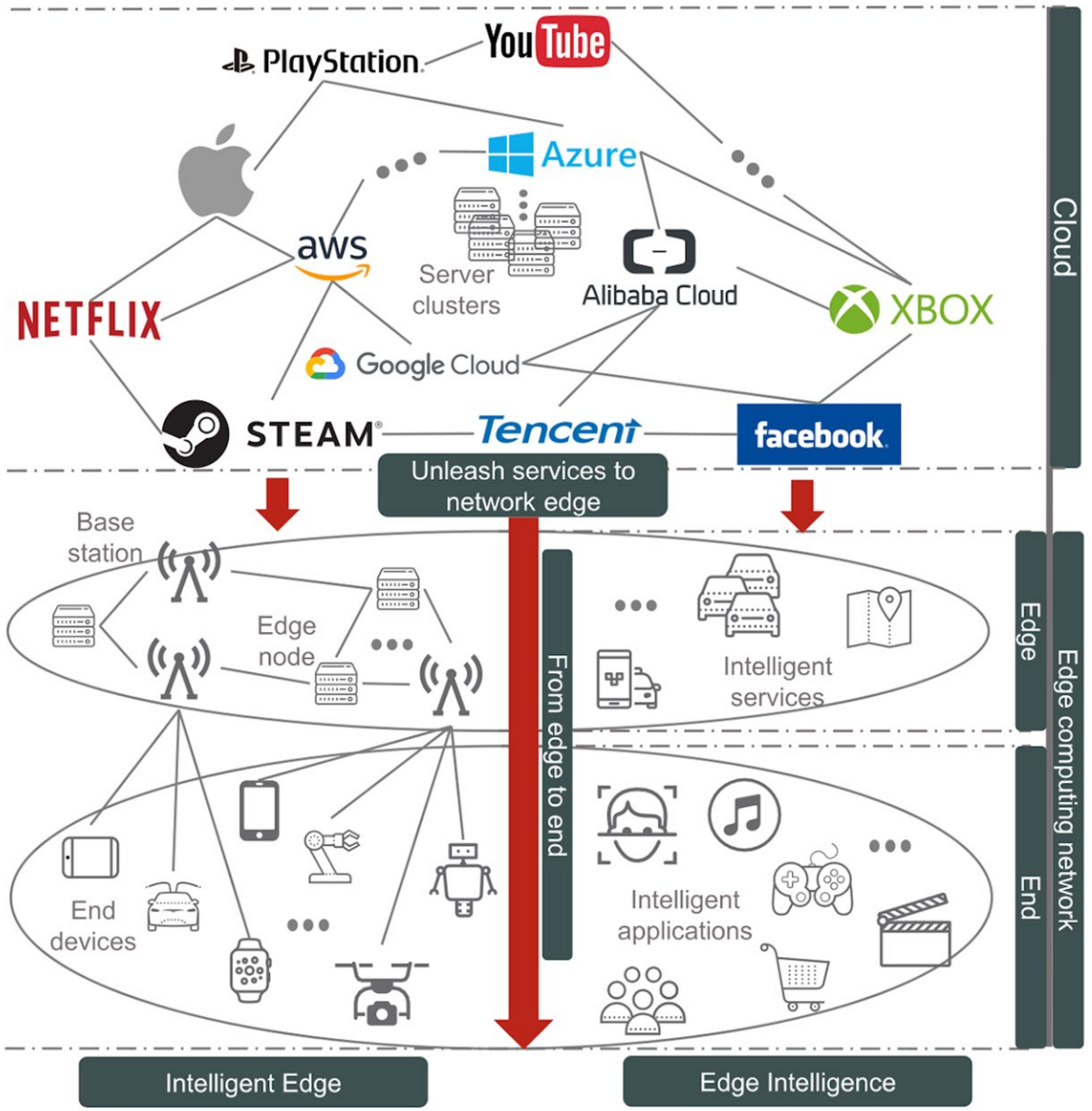
Communication time cannot be neglected

Edge intelligence/intelligent edge

Brings the computation closer

Distributed edge computing

Blockchain



X. Wang, Y. Han, V. C. M. Leung, D. Niyato, X. Yan, and X. Chen, “Convergence of Edge Computing and Deep Learning: A Comprehensive Survey.” *IEEE Communications Surveys & Tutorials*, 2020.

High-definition and real-time Earth observation

Satellites capture images that can be distributed among them for processing

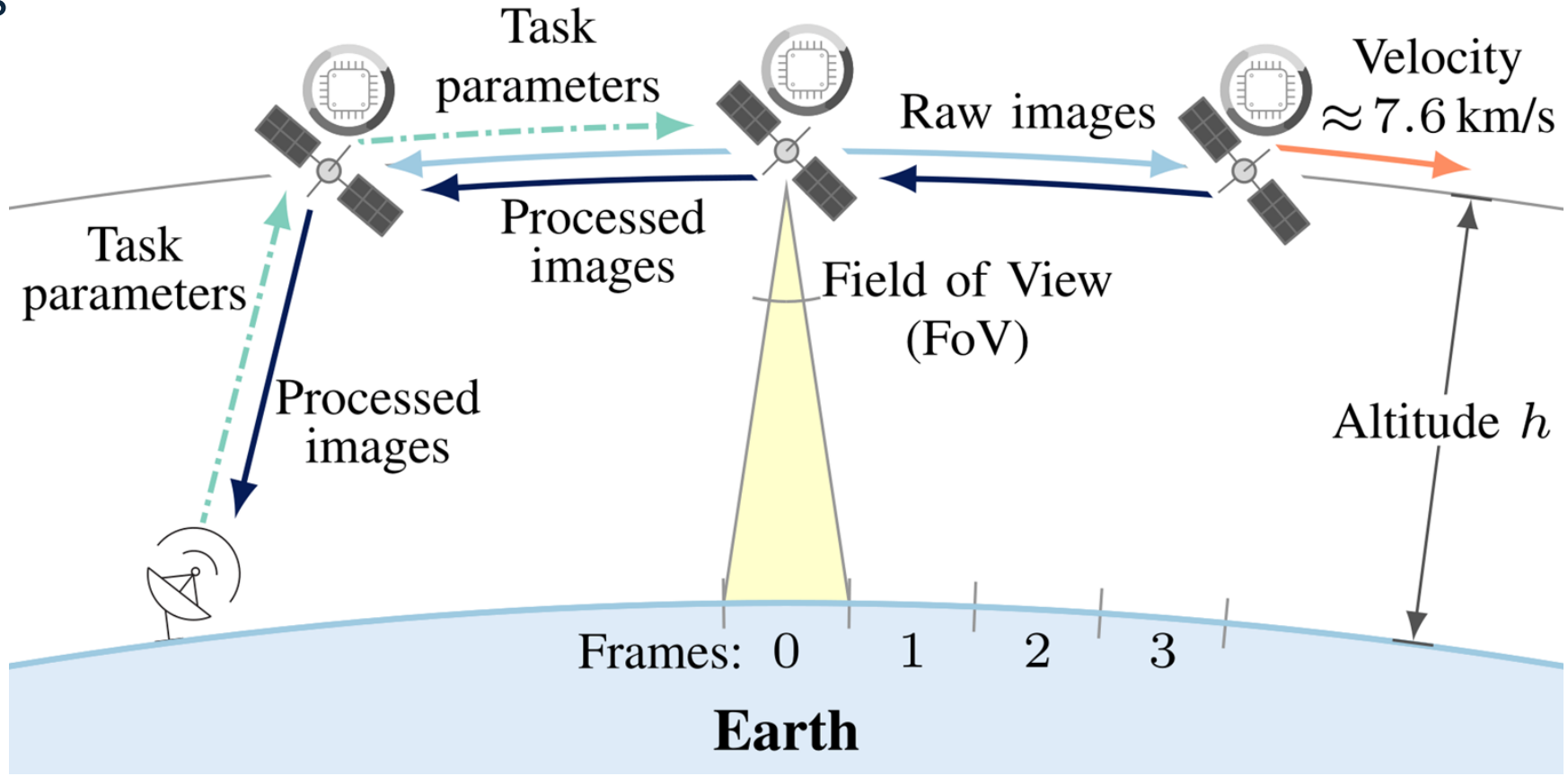
Free-space optical links

- Intra-plane
- High-rate

Ring topology

Constraints

- Processing
- Downlink
- Queueing



High-definition and real-time Earth observation

Process a task to reduce its size while minimizing latency

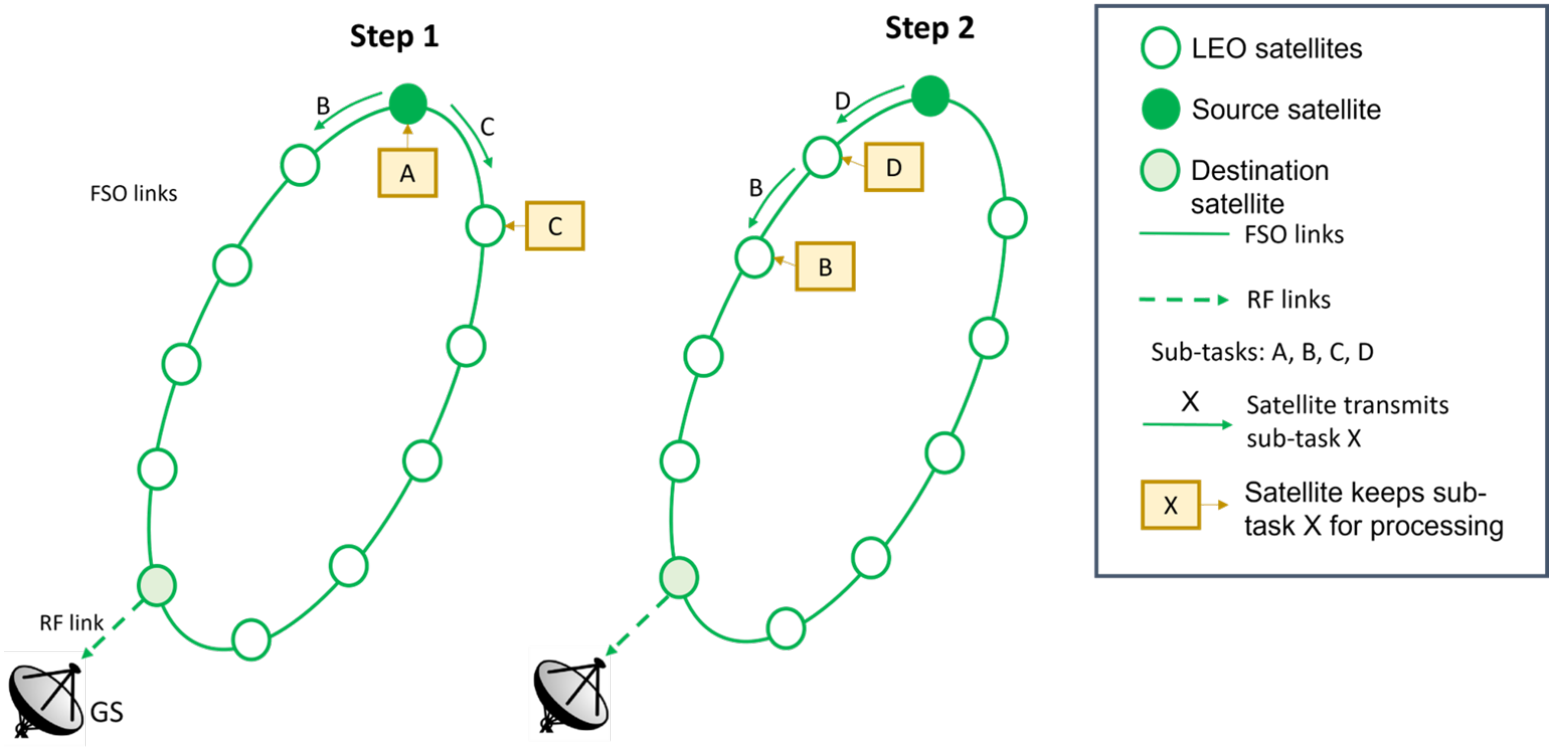
What is the optimal no. of parallel processors p?

Steps:

1. Image segmentation
2. Scatter: example for $p = 4$
3. Parallel processing
4. Gather at the GS

Transmission time

Propagation delay



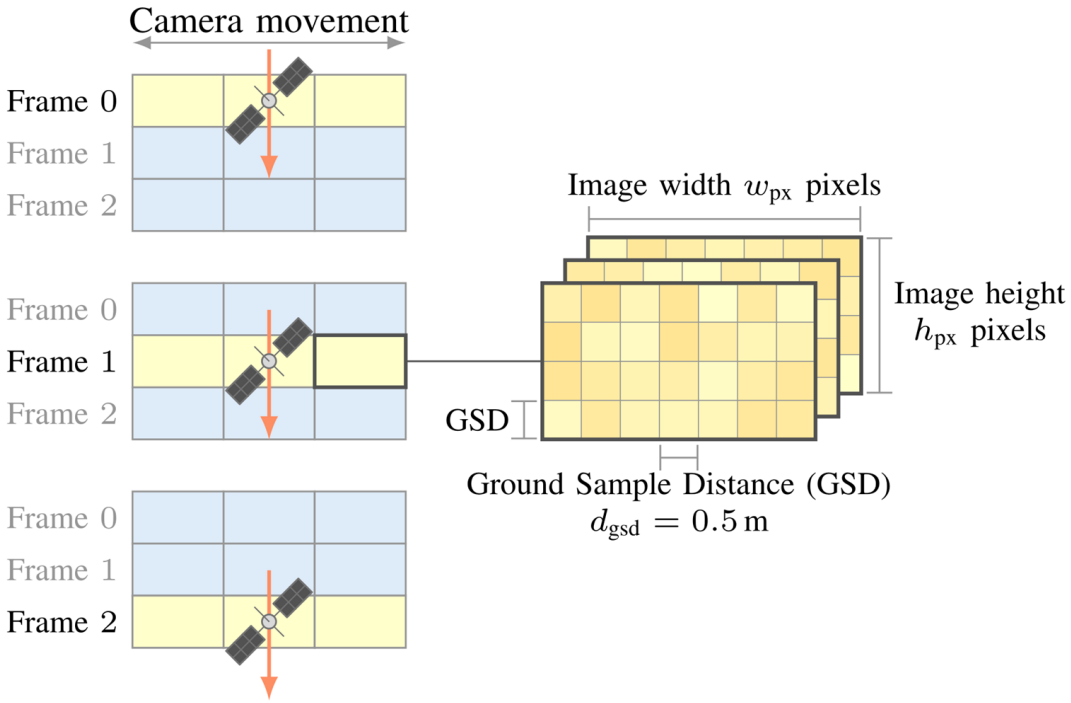
Transmission time

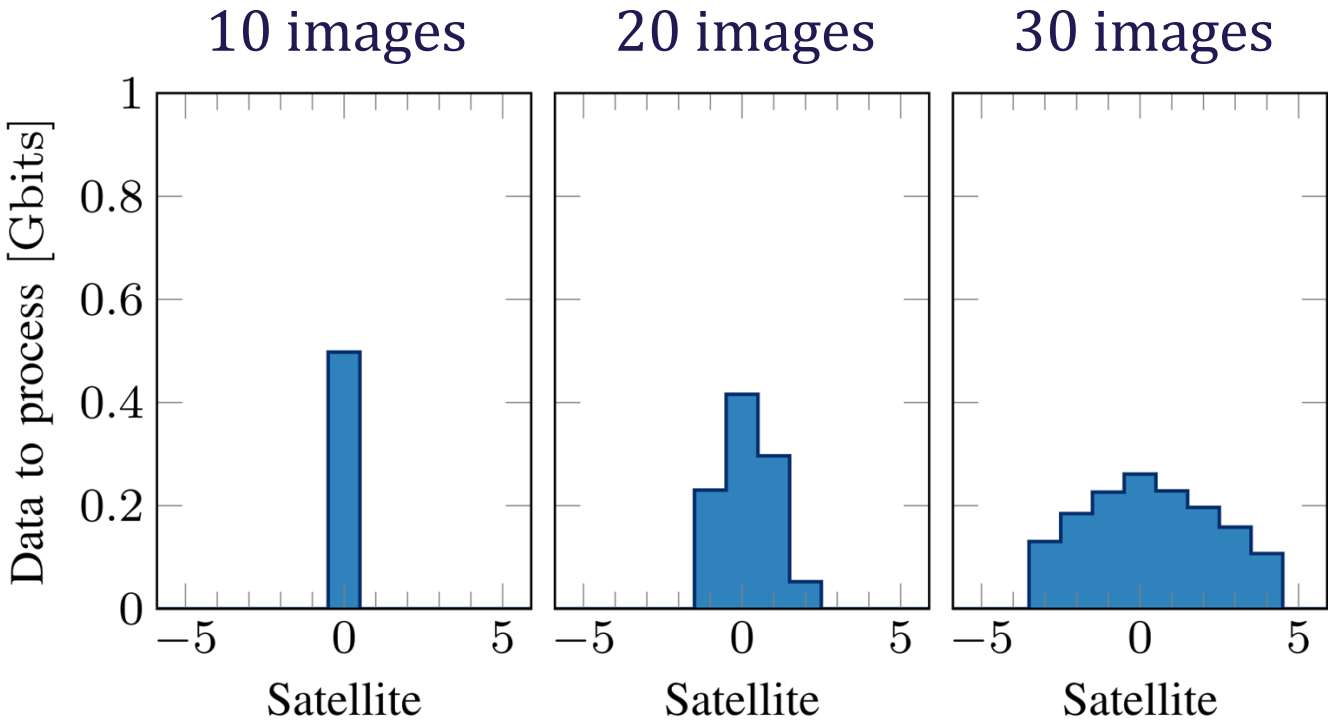
Propagation delay

High-definition and real-time Earth observation

Satellites generate batches of W high-definition images

20 satellites at 600 km with 10 Gbps links and 1.8 GHz quad-core processor





5. Summary and exercises

Summary

Computing architectures have evolved significantly

The perfect network topology balances scalability with transport capacity

Easiest way to parallelize is to assign big and independent tasks

Data segmentation can be useful, but has an overhead

The Message Passing Interface is mostly asynchronous but has tools for synchronization

Distributed computing is a major area of interest:

- Allows to break the barriers of local processing
- Distributed computing and Artificial Intelligence as a service

Exercises

1. Write a code to calculate the average time to broadcast and scatter arrays with 10000 elements using either:

- Send-receive (no broadcast nor scatter)
- Broadcast and scatter

Make a plot to compare their performance for different numbers of processes.

2. Write a code to perform matrix multiplication with MPI: $A \times B = C$. Compare the execution time with a sequential implementation for the same problem.

a. Which one is faster?

b. Which one was easier to code?

c. Which communication operations did you use?

Appendix

All-to-all broadcast in a ring

Desired result $\mathcal{M} = \{m_1, m_2, \ldots, m_p\}$

Naïve implementation: p one-to-all broadcasts

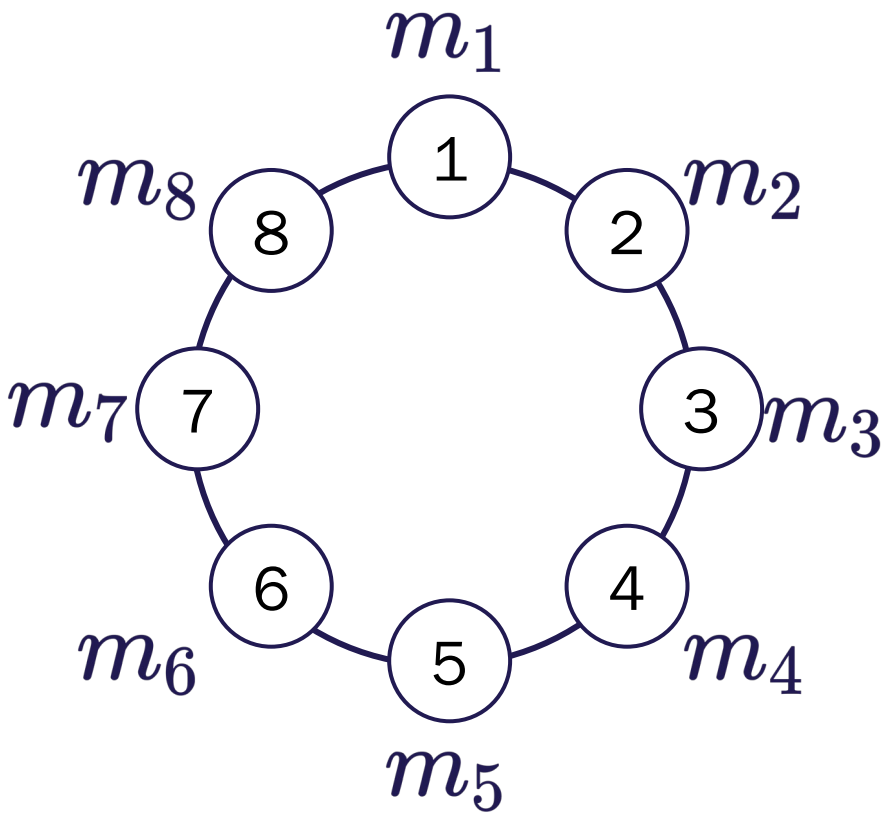
Complete in $p \log_2 p$ steps

But all the links can be used simultaneously

to complete in $p - 1$ steps

Total cost is $t_{\text{comm}} = (t_s + m t_w)(p - 1)$

Used in matrix multiplication

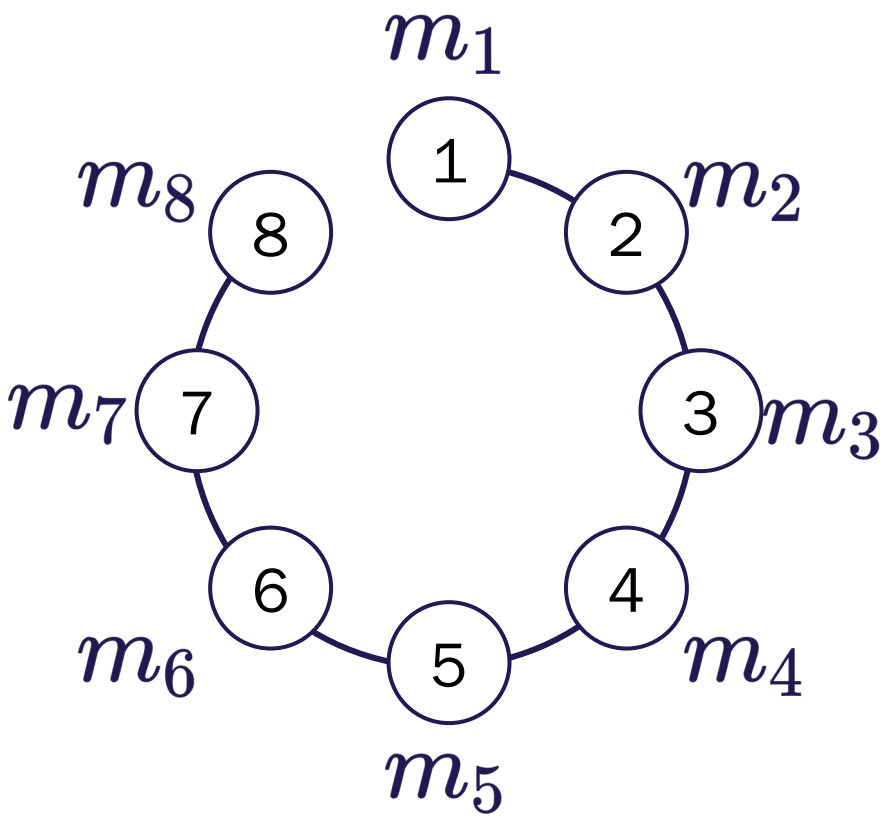


All-to-all broadcast in a linear array

Set of messages to broadcast

$$\mathcal{M} = \{m_1, m_2, \ldots, m_p\}$$

Recall cut-through routing



All-to-all broadcast in a mesh

Set of messages to broadcast $\mathcal{M} = \{m_1, m_2, \ldots, m_p\}$

Completed in two phases

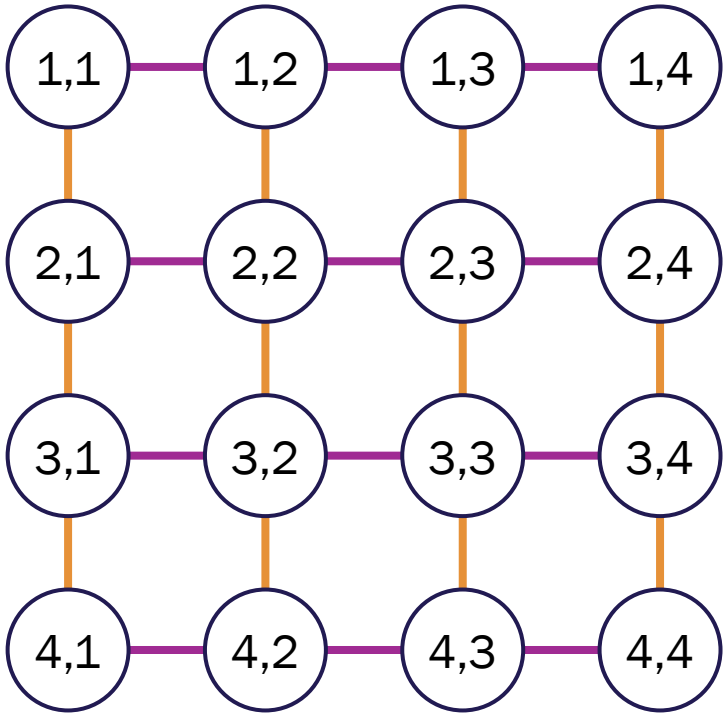
1.Rows: messages of length m

Cost: $t_1 = (t_s + mt_w)\left(\sqrt{p} - 1\right)$

2. Columns: messages of length $m\sqrt{p}$

Cost: $t_2 = \left(t_s + mt_w\sqrt{p}\right)\left(\sqrt{p} - 1\right)$

Total cost: $t_{\text{comm}} = t_1 + t_2 = 2t_s\left(\sqrt{p} - 1\right) + mt_w(p-1)$



Comparison of the communication cost

All-to-all broadcast

Linear and ring topologies: $t_{\text{comm}} = (t_s + mt_w)(p-1)$

Mesh topologies: $t_{\text{comm}} = 2t_s\left(\sqrt{p}-1\right) + mt_w(p-1)$

Hypercube topologies: $t_{\text{comm}} = t_s \log_2 p + mt_w(p-1)$

The term mt_w has the greatest time and is always scaled by $p-1$

The efficiency of these topologies is roughly the same

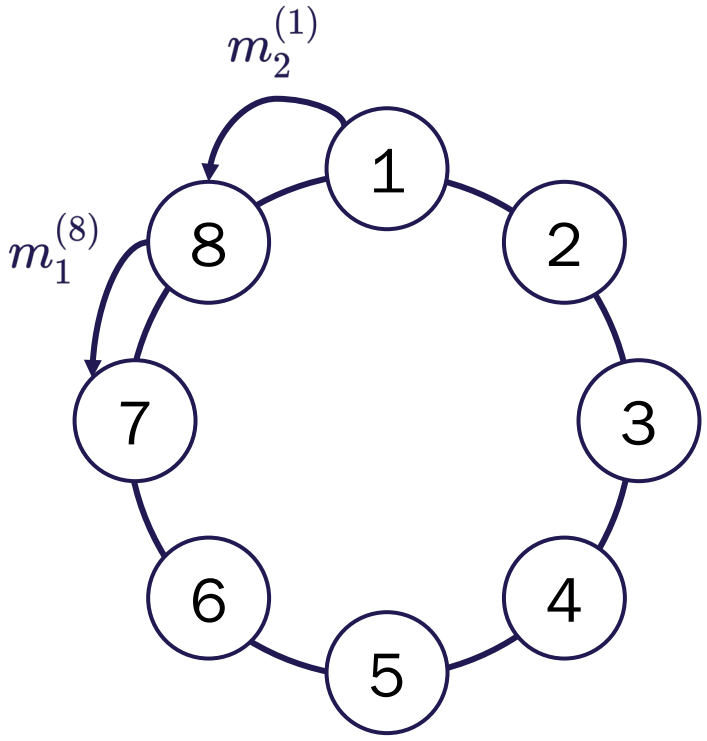
All-to-all reduction in a ring

Set of messages at node u is $\mathcal{M}_u = \left\{ m_1^{(u)}, m_2^{(u)}, \ldots, m_p^{(u)} \right\}$

Processor u requires $\sum_{i=1}^{p} m_i^{(u)}$

Additional operation

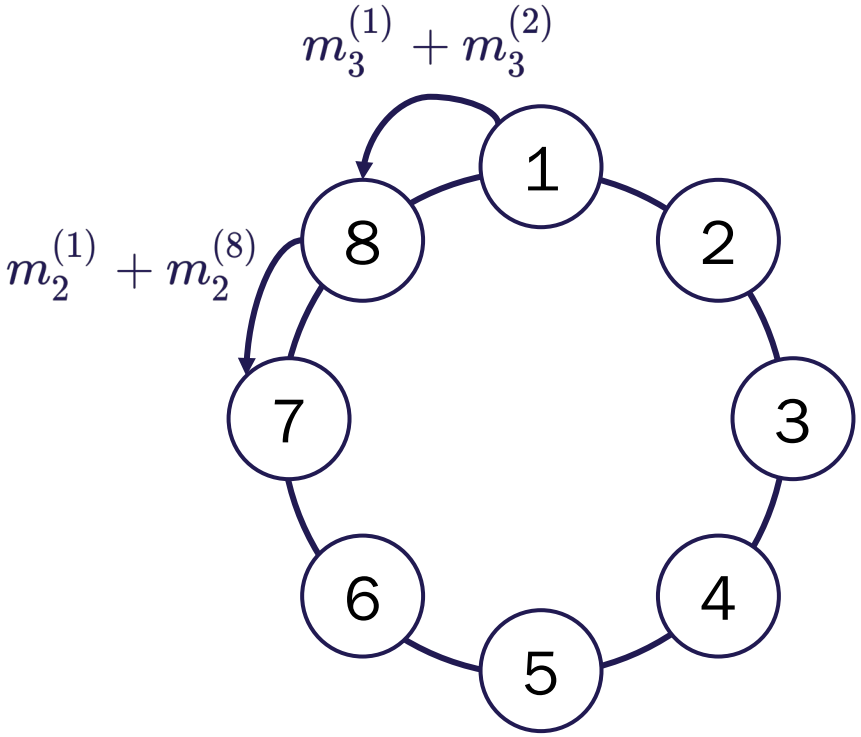
Combine the messages with same destination



All-to-all reduction in a ring

Set of messages at node u is $\mathcal{M}_u = \left\{ m_1^{(u)}, m_2^{(u)}, \ldots, m_p^{(u)} \right\}$

Processor u requires $\sum_{i=1}^{p} m_i^{(u)}$



Additional operation

Combine the messages with same destination

That’s it for today